

Scaling vs. Precision: Mapping the Pareto Frontier of Memory-Bound LLM Inference

Benjamin Bensch^{*}, Theodore Schiffman^{*}, Christian Montgomery^{*}, and Armand Link^{*}

^{*} *Department of Computer Science
Columbia University
New York, United States*

Abstract—This paper presents an approach to determine the optimal tradeoff between parameter count and quantization level. Modern large language model (LLM) deployments are typically bottlenecked by GPU memory, forcing developers to choose between running a larger model at lower numerical precision or a smaller model at higher precision. We visualize this tradeoff by running an inference matrix of the Qwen-2.5 model family at 7B, 14B, 32B, and 72B parameters across four precision levels: BF16, GPTQ-Int8, AWQ-Int4, and BitsandBytes-NF4. Experiments were run on a single 96 GB GPU using the vLLM inference engine. We measure throughput, latency, peak memory, answer quality, and NVIDIA Nsight kernel profiles. Two complementary grids, ShareGPT with GSM8K and random-uniform prompts with MMLU, show that the Pareto frontier is not workload-independent: AWQ dominates the GSM8K frontier at 7B–32B, while BF16 reclaims the throughput end and BitsandBytes the quality end on MMLU. Across both grids, decoding remains memory-bound at every cell, reaching only 4–7% of FP16 peak, which explains why aggressive weight quantization improves throughput despite the 30–35% of GPU time consumed by dequantization kernels.

Index Terms—large language models, quantization, inference, Pareto frontier, vLLM, GPU memory, memory-bound inference

I. INTRODUCTION

While the impact of quantization has been studied extensively, there is significantly less literature exploring how to balance parameter count and numerical precision under hardware constraints. Accordingly, we set out to answer the practical deployment question: what model should be chosen for a fixed VRAM budget?

Large language models (LLMs) have become central to a growing range of applications, including chat personal assistants, code generation, document summarization, and search. However, deploying them in production presents a challenging resource allocation problem. The largest and most capable open-weight models often cannot fit on a single GPU, while smaller models are easier to run but suffer from lower accuracy. Practitioners must therefore decide, under hardware constraints, whether to deploy a larger model at reduced numerical precision or a smaller model at higher precision.

The literature on quantization is extensive, but most published comparisons hold model size constant and vary only the quantization scheme. This leaves open the question that matters most for real-world deployment, particularly for single-GPU settings: across both accuracy and throughput axes, where does the optimal tradeoff lie?

This work answers that question for a single hardware target, the NVIDIA RTX PRO 6000 Blackwell Max-Q with 96 GB of VRAM, and a single model family, Qwen-2.5 at 7B, 14B, 32B, and 72B parameters. We construct two controlled experimental matrices across all combinations of model size and four quantization methods, differing only in workload shape and quality benchmark: ShareGPT prompts paired with GSM8K reasoning, and uniform random prompts paired with MMLU knowledge recall. Each cell is characterized along five axes: end-to-end throughput, answer quality, time-to-first-token latency distribution, GPU power, and kernel-level profiles via NVIDIA Nsight. From these measurements, we construct a Pareto frontier of quality versus throughput under a fixed 96 GB memory budget and explain its shape using hardware-level profiling data.

II. RELATED WORK

Dettmers and Zettlemoyer propose inference scaling laws for low-bit LLM quantization. They investigated the tradeoffs between a larger low-precision model and a smaller high-precision model and investigated which yields more zero-shot accuracy given the same amount of bits available for modeling. The research involves over 35,000 experiments, with inputs set at 16 bits and parameter sizes between 3 bits and 8 bits within the range of 19M to 176B parameters on the BLOOM, OPT, NeoX/Pythia, and GPT-2 language models. Dettmers and Zettlemoyer found there’s a limited scope for improvement in the bit-level scaling tradeoff, which primarily hinges on quantization block size and data type. Their conclusion was that 4 bit precision is almost universally optimal for total model bits and zero-shot accuracy [1]. This helped motivate and support our problem statement: Given a memory budget, should we allocate more parameters but at lower precision or fewer parameters but at higher precision? This paper concentrates on zero-shot accuracy and fixed model bit budgets, in contrast, our paper analyzes the runtime behavior in practice, considering throughput, latency, power, VRAM breakdown, context size, and kernel profiling with a fixed budget of 96 GB for the GPU.

Kim et al. counter Dettmers and Zettlemayers’ claims that quantizing weights to 4 bits is always memory optimal when moving from short zero shot tasks to long generation reasoning models. They looked at 1,700+ inference workloads on AIME25 and GPQA-Diamond, including 4 bit and 8 bit

GPTQ weight quantization, reasoning token budgets from 2k to 30k, majority-vote parallel scaling with up to 16 samples, and KV cache compression through eviction and quantization [2]. Their discovery is a function of scale; Under an effective model size threshold of about an 8 bit 4B model, it’s better to use memory resources to increase model size, whereas beyond this threshold, it’s generally more efficient to allocate memory resources for longer generation lengths and inference compute at test time. Our results are very aligned with this paper’s results. Our results reveal that weight quantization enhances performance and context size, while KV-cache memory becomes the bottleneck resource once larger context lengths or increased concurrency are used. We also found that quantization choice depends on the inference workload being run.

III. METHODOLOGY

The goal of our paper is to investigate the tradeoff between model scale and numerical precision under a fixed 96 GB single-GPU memory budget and specific inference workloads. To streamline model architecture, all the experiments use the Qwen2.5-Instruct family [3] and vary only model size and weight format (quantization). Four models are tested with four different precision levels as follows: 7B, 14B, 32B, and 72B, using BF16, GPTQ-Int8, AWQ-Int4, and BitsAndBytes NF4 respectively. We apply GPTQ and AWQ on pre-quantized Qwen checkpoints, while for BitsAndBytes we apply double quantization with NF4 local materialization on the GPU with BF16 computation. The non-quantized version is denoted by Normal in our grid and reported by vLLM metadata as being BF16. The 72B-BF16 is the only missing configuration due to being too large to fit inside our 96GB VRAM limit.

The hardware used for each experiment and project development is a NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition, which has a capacity of 96GB VRAM. The software stack consists of the following tools: Ubuntu 24.04.4 on WSL2, NVIDIA drivers 582.16, PyTorch 2.10.0+cu128, Transformers 4.57.1, lm-eval-harness 0.4.11, and vLLM. Models are served through vLLM’s OpenAI-compatible server with `dtype=auto`, `trust_remote_code=True`, and the appropriate quantization backend. For the main performance sweeps, vLLM uses a fixed 10 GiB KV-cache pool via `--kv-cache-memory-bytes 10G`, making the static model weight footprint the primary memory variable.

We use two paired benchmark grids for the experiments. The first compares ShareGPT performance benchmarking to GSM8K quality evaluation. ShareGPT makes chatbot style prompts with prompt length variation which mimics real prompts. For each performance run, we use 500 prompts, infinite request rate, and maximum concurrency 64. GSM8K quality evaluation is done with lm-eval-harness using 8-shot prompting, 250 questions, and strict exact-match accuracy metrics, and is focused on grade school mathematics. For the second grid of experiments, we used synthetic random prompts for MMLU quality evaluation. Random prompts have 500 requests, with 512 input tokens and 256 output

tokens per request, at an infinite request rate and maximum concurrency 64. MMLU quality evaluation is done with 5-shot prompting on all 57 subtasks, with five examples per subtask, and aggregate accuracy metrics. Therefore we paired quality and performance benchmarks together, and profiled each run throughout our experiments.

For each experiment run, vLLM bench tracks throughput, time to first token, time per output token, inter-token latency, and end-to-end latency percentiles. Output-token throughput is used as a primary performance metric since it measures sustained autoregressive decoding speed, and is indicative of performance. The profiler running during the experiments constantly samples GPU memory usage, utilization, temperature, and power draw from `nvidia-smi`, while vLLM Prometheus metrics provide KV-cache occupancy, generated-token counters, request queue state, and latency aggregates. The following performance metrics are measured from run traces: peak VRAM usage, average power, energy consumption per generated token, KV-cache expansion, instantaneous decode throughput, and KV-cache/weight-memory ratio.

For each experiment run respective to their workload grid, we then plotted answer quality vs. output-token throughput and classify a run as dominated if there is a second run that obtains higher/equal quality and higher/equal throughput. All nondominated runs constitute the Pareto frontier. We compute a separate Pareto frontier for ShareGPT/GSM8K and random/MMLU, since prefill cost, decode cost, and quantization overhead depend on the workload distribution. We also charted many other aspects of the experiment runs, which you can find in the experimental results section.

Throughout our experimentation, we profile runs using NVIDIA Nsight Systems as well. The kernels are classified into categories of GEMM, dequantization, Flash Attention, KV-cache ops, normalization, sampling, memory operations, and miscellaneous vLLM/Triton kernels. Achieved compute throughput is estimated as $2 \times \text{parameters} \times \text{tokens}$ divided by wall-clock time. Comparing to the GPU’s theoretical FP16 peak, we classify the workload as compute bound or memory bound. We also ran a context capacity test using the vLLM start-up logs and measuring memory allocation using GPU to determine how much extra context length can be achieved because of quantization.

IV. EXPERIMENTAL RESULTS

A. Pareto Frontier

The Pareto frontier for GSM8K begins with 7B-AWQ at approximately 4500 tokens per second and 0.78 accuracy, then proceeds through 7B-GPTQ, 14B-AWQ/GPTQ, and finally 72B-AWQ at approximately 470 tokens per second and 0.95 accuracy. The unquantized BF16 models are dominated almost everywhere, particularly at 32B, where quantized versions at the same model scale achieve similar quality with much higher throughput. As the frontier climbs toward larger models, each additional gain in quality requires a progressively larger throughput sacrifice.

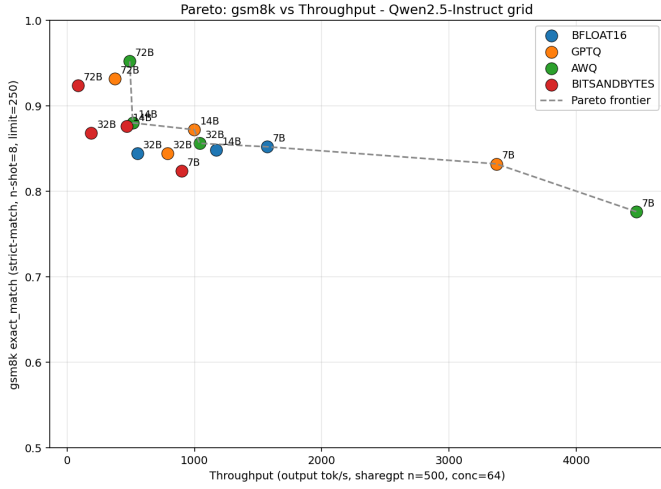


Fig. 1. Pareto frontier of answer quality versus throughput for GSM8K/sharegpt workloads.

For MMLU accuracy, we observe a nearly linear scaling across model sizes from roughly 0.78 at 7B to 0.90 at 72B-BnB. This corresponds to a 4–5% accuracy gain per model doubling.

B. Accuracy by Model Size

Strict-match accuracy on 250 GSM8K questions ranges from 0.776 for 7B-AWQ to 0.952 for 72B-AWQ. Quality is dominated by parameter count: every 72B configuration scores at least 0.92 regardless of quantization, while the 7B configurations cluster between 0.78 and 0.85. Quantization does not substantially reduce accuracy for 14B and larger models on both Pareto charts. However, at 7B, AWQ loses roughly 8% accuracy on GSM8K versus only 1.5% on MMLU, suggesting that activation-aware INT4 quantization weakens reasoning-chain accuracy more than knowledge recall.

Notably, for GSM8K, BF16 accuracy is nearly flat across 7B, 14B, and 32B, moving from 0.852 to 0.848 to 0.844. This suggests that GSM8K accuracy for Qwen-2.5 saturates below the 72B regime, which helps explain why the Pareto frontier rewards a quantized large model over a full-precision mid-sized model.

C. Decode Throughput

Output throughput differs by more than an order of magnitude across the grid, from 4474 tokens per second for 7B-AWQ to 83 tokens per second for 72B-BnB. Within each model size, AWQ and GPTQ are substantially faster than BitsandBytes, with BF16 typically falling between INT8 and INT4-AWQ. Throughput approximately halves with each doubling of model size, consistent with a memory-bound regime where clock time is dominated by high-bandwidth memory reads rather than matrix multiplication.

BitsandBytes pays a consistent throughput penalty across every model size, roughly $2\times$ slower than AWQ at the same

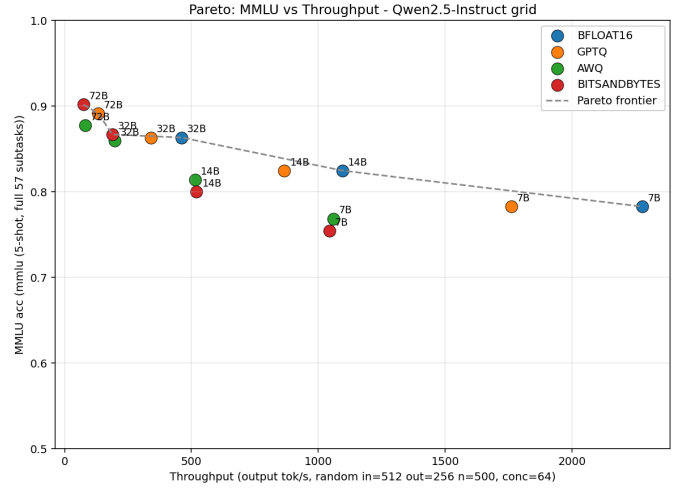


Fig. 2. Pareto frontier of answer quality versus throughput for MMLU/random workloads.

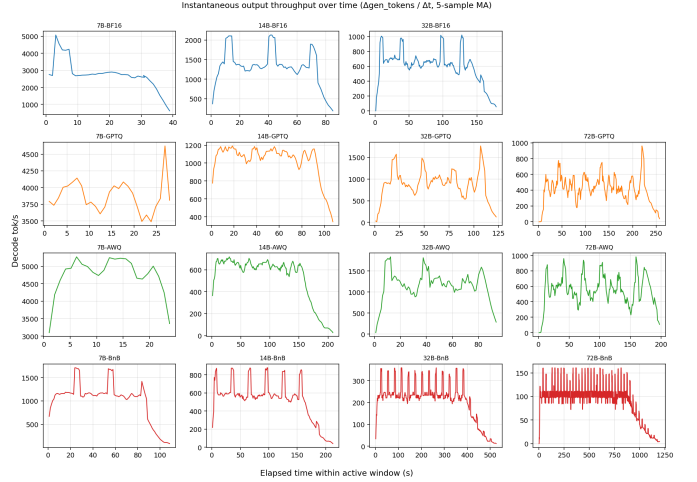


Fig. 3. Instantaneous decode throughput over time. Replace this placeholder with the final throughput-stability chart.

scale. This likely reflects eager FP16 dequantization inside matrix multiplication rather than the kernel-fused dequantization used by AWQ and GPTQ.

D. Throughput Stability

Tracking instantaneous decode throughput, computed as change in output tokens divided by change in time and smoothed over five-second windows, shows that headline tokens-per-second values conceal meaningful temporal variability. Most configurations exhibit a 5–10 second warmup ramp followed by a steady plateau. Larger BitsandBytes configurations, especially 32B-BnB and 72B-BnB, oscillate by approximately $\pm 30\%$ throughout the run. This appears to be a signature of preemption events as the KV pool fills near capacity and vLLM evicts requests. AWQ and GPTQ at the same scale maintain flatter throughput because their faster decode prevents the KV pool from saturating as quickly.

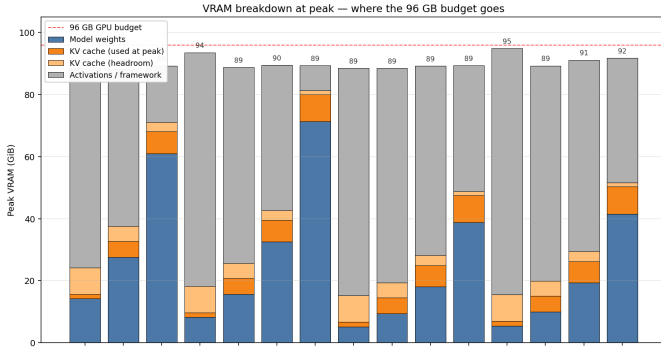


Fig. 4. Peak VRAM composition and KV-cache-to-weight ratio across model sizes and quantization levels. Replace this placeholder with the final VRAM chart.

E. Composition of VRAM

All runs were configured with vLLM’s KV pool fixed at 10 GB, so the main memory variance across cells is the weight footprint plus allocator overhead. Peak VRAM of approximately 90 GB is a nearly constant ceiling set by vLLM startup allocation. Quantization does not reduce that ceiling directly; instead, it changes the composition of memory allocated to model weights versus the allocator stack.

The peak KV-to-weight ratio jumps from 0.10 for 7B-BF16 to 0.55 for 14B-AWQ because the denominator shrinks under quantization. Per-token KV remains FP16, and absolute KV usage is roughly constant per model size. This makes KV the bottleneck that would bind first under longer contexts or higher concurrency. The peak KV-to-weight ratio reaches its maximum at 14B in each quantization family because Qwen-2.5 doubles its grouped-query attention KV heads from 4 at 7B to 8 at 14B and above. This one-time architectural jump inflates per-token KV faster than weights at that step. For 32B and 72B, the KV-head count stays flat while weights continue scaling, so the ratio declines.

F. Throughput versus Power

GPU power draw ranges from 178 W to 292 W, compared with the card’s 300 W thermal design power. This indicates that, on the lower end, streaming multiprocessors are frequently stalled waiting on memory rather than executing matrix multiplications. The throughput-versus-power Pareto front is anchored by 7B-AWQ at approximately 4500 tokens per second and 187 W, or 24 tokens per joule. At the opposite extreme, 72B-BnB reaches approximately 80 tokens per second at 290 W, or 0.3 tokens per joule. This corresponds to an approximately 80× efficiency gap.

G. Time to First Token

Time to first token (TTFT) is the wall-clock delay from request submission to the first generated token. The TTFT distribution spans roughly 50 ms to 25 s across percentiles. For several runs, tail behavior differs substantially from mean

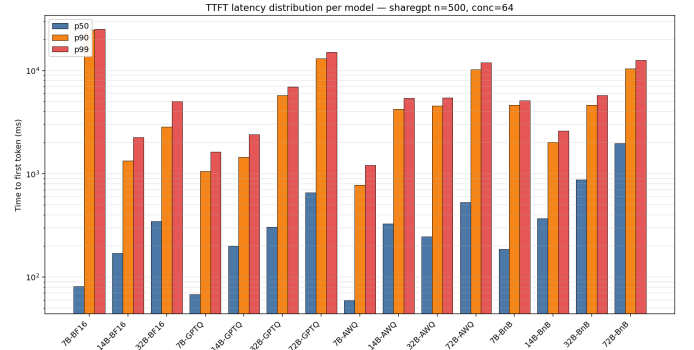


Fig. 5. Time-to-first-token latency by model and quantization level.

TABLE I
APPROXIMATE ARITHMETIC INTENSITY BY PRECISION

Precision	FLOPs	Bytes	Intensity
BF16	2W	2W	1
INT8	2W	1W	2
INT4	2W	0.5W	4

TTFT, especially for 7B-BnB and 72B-GPTQ. Smaller aggressively quantized models maintain near-real-time latency performance, while all tested 32B and 72B models reach multiple seconds at p99, which would be unacceptable for realistic chatbot workloads.

H. Roofline Analysis

Roofline analysis shows each AWQ model’s achieved compute throughput using the approximation in (1):

$$\text{Achieved FLOPs} = \frac{2 \cdot \text{parameters} \cdot \text{tokens}}{\text{wall-clock time}}. \quad (1)$$

This is counted for one transformer forward pass and compared with Blackwell’s peak FP16 tensor core capacity of 290 TFLOPS. Each AWQ-quantized model reaches only 4–7% of peak, leaving tensor cores idle for most of the run. This indicates that most GPU energy is spent waiting on HBM rather than computing. Each generated token still requires reading the full set of weights from high-bandwidth memory before matrix multiplication can occur.

The key driver is arithmetic intensity, the ratio of FLOPs to bytes read from HBM. If the GPU is bottlenecked on bytes per second rather than compute, shrinking weights through quantization translates directly into throughput. Table I summarizes the approximate arithmetic-intensity relationship across precision levels.

I. GPU Time by Kernel Category

Breaking total GPU time by kernel category for AWQ models exposes the hidden cost of quantization. AWQ de-quantization kernels alone consume approximately 30% of GPU time across all four model sizes, second only to matrix multiplication. AWQ stores weights in packed 4-bit form, but tensor cores operate on 16-bit values. Each layer must

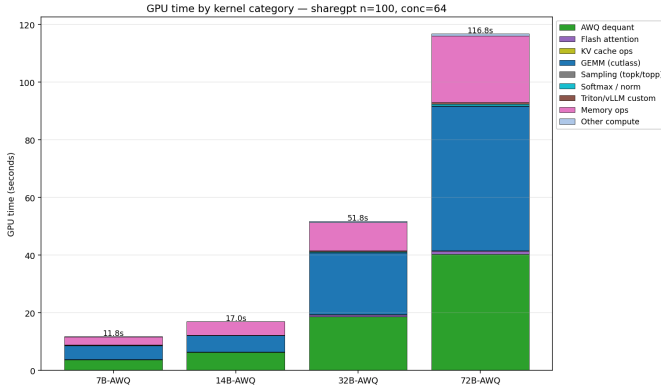


Fig. 6. GPU time by kernel category for AWQ models.

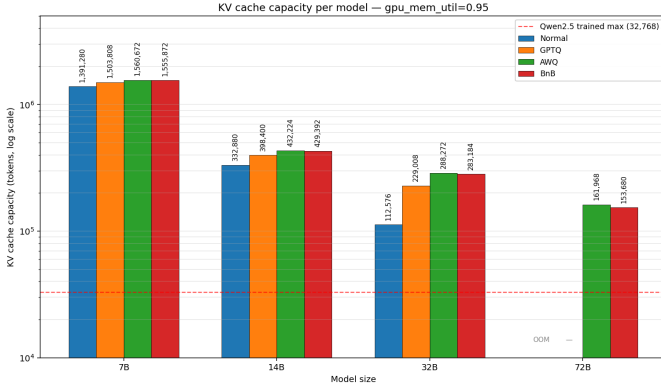


Fig. 7. Maximum supported single-request context length across quantization levels and model sizes..

therefore dequantize by unpacking and rescaling 4-bit values into FP16 before GEMM can run, adding overhead on every forward pass.

Thus, the throughput gains from quantization involve a tradeoff: fewer bytes cross the HBM bottleneck, but an additional dequantization step is required at every layer. This explains why 7B-AWQ reaches approximately 4500 tokens per second rather than the 6000 tokens per second that a naive $4\times$ weight-shrink calculation might predict from the approximately 1500 tokens per second FP16 baseline.

J. Context Capacity

The maximum supported context length varies by more than an order of magnitude across the grid, so the context-capacity chart uses a log scale. 7B-AWQ reaches roughly 1.5 million tokens of context, while the 72B model at 16-bit full precision does not fit into the 96 GB budget. Smaller, more aggressively quantized configurations consistently support longer contexts. Each gigabyte of memory saved by quantizing weights becomes available for the KV cache, which directly limits how long a single context can grow. This demonstrates that quantization improves not only throughput but also deployable context length.

V. DISCUSSION

Our work aimed to answer a seemingly simple practical question: given a fixed 96 GB GPU memory budget, which combination of model size and weight quantization should be deployed? Our goal was to calculate a Pareto frontier showing the tradeoff between throughput and model accuracy across Qwen model sizes and quantization levels.

The clearest cross-grid finding is that the Pareto frontier is sensitive to both workload and quality metric. On GSM8K with ShareGPT prompts, the frontier passes through three of the four model sizes via AWQ: 7B-AWQ, 14B-AWQ, 32B-AWQ, and 72B-AWQ. The 14B-AWQ and 32B-AWQ configurations occupy complementary corners, with 514 tokens per second at 0.880 accuracy versus 1040 tokens per second at 0.856 accuracy. On MMLU with random prompts, BF16 reasserts itself at the throughput end, with 7B-BF16 dominating every 7B-quantized variant on both accuracy and throughput when input prefill is heavy. BitsandBytes takes the quality end with 72B-BnB at 0.902 MMLU accuracy. For a deployment with balanced throughput-quality preferences on chatbot-shaped workloads, 14B-AWQ remains the strongest default. For prefill-heavy batch summarization workloads, 7B-BF16 wins. For accuracy-critical use cases, 72B-BnB is preferable.

The hardware-level profiling from Nsight explains why the frontier takes this shape. During autoregressive decoding, the GPU is memory-bound. Tensor cores reach only 4–7% of theoretical peak FP16 throughput because every generated token first requires reading the model’s entire set of weights from GPU high-bandwidth memory before arithmetic can be performed. Quantization helps directly by shrinking the bytes per weight that must cross HBM at each step. However, AWQ dequantization kernels consume 30–35% of GPU time, which is why AWQ throughput gains are closer to $3\times$ than the naive $4\times$ weight-shrink expectation.

Peak VRAM consumption is not a useful discriminator in our experiments because every configuration converges to roughly 89 GB peak. All runs were configured with vLLM’s KV pool fixed at 10 GB, so the main memory variance across cells is the weight footprint plus allocator overhead. vLLM’s allocator holds onto memory pages acquired during startup, meaning quantization does not reduce the observed peak VRAM ceiling directly. Instead, it changes the composition of the memory allocation: lower-precision weights occupy less of the fixed allocation, leaving more effective room for the KV cache and allocator stack. This distinction matters because peak VRAM alone can make quantization appear less useful than it is, even though the underlying memory composition and achievable context capacity change substantially.

These takeaways are specific to our experimental setup: a single inference workload shape, a fixed 10 GB KV cache pool, 64 concurrent requests, and answer quality benchmarked with GSM8K and MMLU rather than a broader evaluation suite. The most important caveat for future experiments is that, as quantization shrinks weights, the KV cache becomes

the binding memory constraint under longer contexts or higher concurrency. Further weight quantization alone cannot reduce that constraint. A natural extension of this work is to enable FP8 KV cache and rerun the same grid, which would likely shift the Pareto frontier further to the right and unlock substantially longer-context deployments within the same 96 GB VRAM constraint.

VI. CONCLUSION

This experiment set out to answer: at a fixed 96 GB GPU memory budget, what combination of model size and weight precision produces the best inference outcome, and where does the optimal tradeoff lie on the size-versus-precision plane?

Across a controlled experimental matrix of Qwen-2.5 at 7B, 14B, 32B, and 72B parameters paired with BF16, GPTQ-Int8, AWQ-Int4, and BitsandBytes-NF4 quantization, we created a Pareto frontier of answer quality versus generation throughput. We found that the frontier shape depends on workload and quality metric, with AWQ dominating short-prompt reasoning workloads and BF16 or BitsandBytes taking over for prefill-heavy knowledge workloads. Hardware-level profiling with Nsight demonstrated the underlying mechanism: decoding is memory-bound at every cell of our grid, which is why aggressive weight quantization improves throughput even after accounting for the added 30–35% of GPU time consumed by on-the-fly dequantization.

The natural extension is to quantize the KV cache to FP8, which our analysis suggests would shift the frontier further to the right by relieving the constraint that becomes binding under longer contexts and higher concurrency. In conclusion, under a fixed VRAM budget on a memory-bound GPU, the best practical strategy is to choose the smallest model that meets the quality threshold and then push quantization as far as possible to maximize the accuracy-throughput tradeoff.

AI USAGE DISCLOSURE

We used AI to assist with many facets of this project.

Tools used: Claude Code, Codex, Claude, Gemini, ChatGPT
Purposes:

- Claude Code was used extensively to create the HPML Model Manager GUI, which was the main tool we used to execute all procedural facets of the project.
- Codex was used for code review.
- Claude, Gemini and ChatGPT were used to (1) find relevant published research on our project topic and (2) research and assist in selecting the LLM serving platform, quantization packages and performance benchmarks.

Questions/Sections Affected: The final presentation and written report are our own work, with the exception of the included charts, which were created using AI written code, some guidance, polished prose, and formatting.

How we verified correctness:

- We tested functionality and inspected output.
- We ran an adversarial code review with Codex using this prompt:

```
/codex:adversarial-review --base
main Correctness-only review.
Verify whether this code performs
its stated intent. Specifically,
review all code for the GUI, and
verify that execution matches
the labels and options in the
GUI. Review all plotting code
and the code that generates the
data used by the plotting code to
verify the data being generated and
plotted matches the intent. This
review should not cover any other
topics, including security, race
conditions, style, maintainability,
performance, dependency issues, and
speculative edge cases unless they
directly cause incorrect output.
Read all files in the project
directory (not just code) to infer
the intended behavior. Produce a
report with: 1. Summary verdict,
2. Intended behavior, 3. Evidence
checked, 4. Correctness issues, 5.
Output mismatches. Do not modify
files.
```

By submitting this assignment, we confirm that the analysis, interpretations, and conclusions are our own, and that any AI assistance is fully disclosed above.

APPENDIX

Energy per Output Token

Energy per output token was computed as average GPU power multiplied by wall duration and divided by total output tokens. 7B-AWQ achieves approximately 0.04 J/token, compared with 3.51 J/token for 72B-BnB. This is an approximately 90× efficiency gap dominated by throughput, since power varies only by about 1.6× across configurations.

KV Cache Growth Over Time

KV-pool occupancy was sampled at 1 Hz throughout each performance run. All 15 runs show a sawtooth-on-rising-baseline pattern: KV usage grows as new requests prefill and drops as completed requests free their KV cache. Larger models with smaller KV headroom, especially 32B-BnB and 72B-BnB, sustain higher peak occupancy. This near-saturation triggers the throughput oscillations visible in the temporal-stability chart.

REFERENCES

- [1] T. Dettmers and L. Zettlemoyer, “The case for 4-bit precision: k-bit inference scaling laws,” *arXiv preprint arXiv:2212.09720*, 2022.
- [2] J. Kim, E. Ewer, T. Moon, J. Park, and D. Papailiopoulos, “Not all bits are equal: Scale-dependent memory optimization strategies for reasoning models,” *arXiv preprint arXiv:2510.10964*, 2025.
- [3] Qwen Team, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2025.